

The `model` field allows you to specify which AI model to use for a specific agent. This enables you to optimize agent performance by selecting models that are best suited for particular tasks - for example, using a code-specialized model for development agents or a reasoning-focused model for analysis tasks.

If not specified, the agent will use the default model configured in your Kiro CLI settings. If the specified model is not available, the agent will fall back to the default model and display a warning message.

Objective

Learn how to configure different models for specific agents to optimize performance for different use cases.

Instructions

Step 1: Check Available Models

1. Start a regular Kiro chat session

```
1 kiro-cli chat
```



2. Check available models

```
1 /model
```



Expected output:

```
Select model (type to search):
> * claude-opus-4.6      2.20x credits  The latest Claude Opus model with 1M context window
  auto                  1.00x credits  Models chosen by task for optimal usage and consistent quality
  claude-sonnet-4.6     1.30x credits  The latest Claude Sonnet model with 1M context window
  claude-opus-4.5       2.20x credits  The Claude Opus 4.5 model
  claude-sonnet-4.5     1.30x credits  The Claude Sonnet 4.5 model
  claude-sonnet-4       1.30x credits  Hybrid reasoning and coding for regular use
  claude-haiku-4.5      0.40x credits  The latest Claude Haiku model
  deepseek-3.2          0.25x credits  Experimental preview of DeepSeek V3.2
  minimax-m2.5          0.25x credits  Experimental preview of MiniMax M2.5
  minimax-m2.1          0.15x credits  Experimental preview of MiniMax M2.1
  glm-5                 0.50x credits  Experimental preview of GLM-5
  qwen3-coder-next      0.05x credits  Experimental preview of Qwen3 Coder Next
```

3. Exit the session

```
1 /quit
```



Step 2: Create Agent with High-Performance Model

```
1 cat > .kiro/agents/code-reviewer.json << 'EOF'
2 {
3   "name": "code-reviewer",
4   "description": "Code review agent using Claude Sonnet 4.5 for detailed analysis",
5   "prompt": "You are an expert code reviewer. Provide thorough, constructive feedback on code quality, security, performance, and best practices. Focus on actionable improvements.",
6   "model": "claude-sonnet-4.5",
7   "tools": [
8     "read",
9     "write"
10  ],
11   "allowedTools": [
12     "read"
13  ]
14 }
15 EOF
```



Step 3: Create Agent with Fast Model

```
1 cat > .kiro/agents/quick-helper.json << 'EOF'
2 {
3   "name": "quick-helper",
4   "description": "Fast response agent using Claude Haiku 4.5 for quick tasks",
5   "prompt": "You are a helpful assistant optimized for quick, concise responses. Provide direct answers and brief explanations.",
6   "model": "claude-haiku-4.5",
7   "tools": [
8     "read",
9     "write"
10  ],
11   "allowedTools": [
12     "read"
13  ]
14 }
15 EOF
```



Step 4: Test Model-Specific Agents

1. Test the code reviewer agent

```
1 kiro-cli chat --agent code-reviewer
```



2. Check which model is being used

```
1 /model
```



Expected:

```
[code-reviewer] > /model

Select model (type to search):
  claude-opus-4.6      2.20x credits  The latest Claude Opus model with 1M context window
  auto                  1.00x credits  Models chosen by task for optimal usage and consistent quality
  claude-sonnet-4.6     1.30x credits  The latest Claude Sonnet model with 1M context window
  claude-opus-4.5       2.20x credits  The Claude Opus 4.5 model
> * claude-sonnet-4.5   1.30x credits  The Claude Sonnet 4.5 model
  claude-sonnet-4       1.30x credits  Hybrid reasoning and coding for regular use
  claude-haiku-4.5      0.40x credits  The latest Claude Haiku model
  deepseek-3.2          0.25x credits  Experimental preview of DeepSeek V3.2
  minimax-m2.5          0.25x credits  Experimental preview of MiniMax M2.5
  minimax-m2.1          0.15x credits  Experimental preview of MiniMax M2.1
  glm-5                 0.50x credits  Experimental preview of GLM-5
  qwen3-coder-next      0.05x credits  Experimental preview of Qwen3 Coder Next
```

3. Test with a code review task

```
"Please review this simple Python function for any improvements:

def calculate_total(items):
    total = 0
    for item in items:
        total = total + item['price']
    return total
"
```

4. Exit and test the quick helper

```
1 /quit
```



```
1 kiro-cli chat --agent quick-helper
```



5. Check the model

```
1 /model
```



Expected:

```
[quick-helper] > /model

Select model (type to search):
  claude-opus-4.6      2.20x credits  The latest Claude Opus model with 1M context window
  auto                  1.00x credits  Models chosen by task for optimal usage and consistent quality
  claude-sonnet-4.6     1.30x credits  The latest Claude Sonnet model with 1M context window
  claude-opus-4.5       2.20x credits  The Claude Opus 4.5 model
  claude-sonnet-4.5     1.30x credits  The Claude Sonnet 4.5 model
  claude-sonnet-4       1.30x credits  Hybrid reasoning and coding for regular use
> * claude-haiku-4.5    0.40x credits  The latest Claude Haiku model
  deepseek-3.2          0.25x credits  Experimental preview of DeepSeek V3.2
  minimax-m2.5          0.25x credits  Experimental preview of MiniMax M2.5
  minimax-m2.1          0.15x credits  Experimental preview of MiniMax M2.1
  glm-5                 0.50x credits  Experimental preview of GLM-5
  qwen3-coder-next      0.05x credits  Experimental preview of Qwen3 Coder Next
```

6. Test with a quick question

```
"What's the difference between a list and a tuple in Python?"
```

Step 5: Test Invalid Model Fallback

```
1 cat > .kiro/agents/fallback-test.json << 'EOF'
2 {
3   "name": "fallback-test",
4   "description": "Agent with invalid model to test fallback behavior",
5   "prompt": "You are a test agent to demonstrate model fallback.",
6   "model": "non-existent-model-123",
7   "tools": [
8     "read"
9   ]
10 }
11 EOF
```



1. Start the agent with invalid model

```
1 kiro-cli chat --agent fallback-test
```



Expected warning message about model fallback:

```
Warning: Model 'non-existent-model-123' not available, falling back to default model
```

2. Verify fallback model

```
1 /model
```



Expected:

```
[fallback-test] > /model

Select model (type to search):
> * claude-opus-4.6      2.20x credits  The latest Claude Opus model with 1M context window
  auto                  1.00x credits  Models chosen by task for optimal usage and consistent quality
  claude-sonnet-4.6     1.30x credits  The latest Claude Sonnet model with 1M context window
  claude-opus-4.5       2.20x credits  The Claude Opus 4.5 model
  claude-sonnet-4.5     1.30x credits  The Claude Sonnet 4.5 model
  claude-sonnet-4       1.30x credits  Hybrid reasoning and coding for regular use
  claude-haiku-4.5      0.40x credits  The latest Claude Haiku model
  deepseek-3.2          0.25x credits  Experimental preview of DeepSeek V3.2
  minimax-m2.5          0.25x credits  Experimental preview of MiniMax M2.5
  minimax-m2.1          0.15x credits  Experimental preview of MiniMax M2.1
  glm-5                 0.50x credits  Experimental preview of GLM-5
  qwen3-coder-next      0.05x credits  Experimental preview of Qwen3 Coder Next
```

Verification

- ☒ Different agents use their specified models
- ☒ Model selection affects response style and capabilities
- ☒ Invalid models fall back gracefully to default
- ☒ `/model` command shows current model for each agent

Key Learnings

- `model` field allows per-agent model selection
- Choose models based on agent's specific use case
- Fast models (Haiku) for quick tasks, powerful models (Sonnet) for complex analysis
- Invalid models automatically fall back to default with warning
- Model selection can significantly impact agent performance and response quality